Experiment 5: Hardware Q-Learning Agent

FPGA Build Challenge 2026 – Advanced level – AMD PYNQ-Z2 (Zynq-7000 XC7Z020)

1. Objective

Reinforcement learning (RL) trains an agent to make decisions by interacting with an environment and receiving rewards. The objective of this experiment is to build an entire Q-learning agent in the FPGA fabric: the maze environment, the Q-table, the Bellman update arithmetic, the exploration strategy and the training state machine all run in hardware. The ARM processor only sets the hyperparameters and reads the results over AXI4-Lite. The agent learns to solve an 8×8 maze from scratch, with no prior knowledge of the layout, and finds the shortest path in under 5 ms of wall-clock time.

2. Background: Q-Learning

Q-learning is a model-free, off-policy RL algorithm. It maintains a table Q(s, a) of expected future reward for each state–action pair. After every step, one entry is updated with the Bellman equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot [\ \text{reward} + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)\ ]$$

where α is the learning rate (how fast the agent updates its beliefs), γ is the discount factor (how much it cares about future rewards), and s' is the state reached after taking action a. The agent follows an ε-greedy policy: with probability ε it picks a random action (exploration), otherwise the best known action (exploitation). ε starts high and decays after each episode, so the agent gradually shifts from exploring to exploiting.

Parameter	Value	Fixed-point representation
α (learning rate)	0.10	0x1A (0.8 unsigned: 26/256 ≈ 0.1016)
γ (discount factor)	0.95	0xF3 (0.8 unsigned: 243/256 ≈ 0.9492)
ε (initial exploration)	0.90	0xE6 (0.8 unsigned: 230/256 ≈ 0.8984)
Rewards	+100 goal, −10 wall, −1 step	Signed 8.8 fixed point (16 bits)
Episodes	2000	32-bit unsigned
Max steps per episode	256	16-bit unsigned

3. Block Diagram

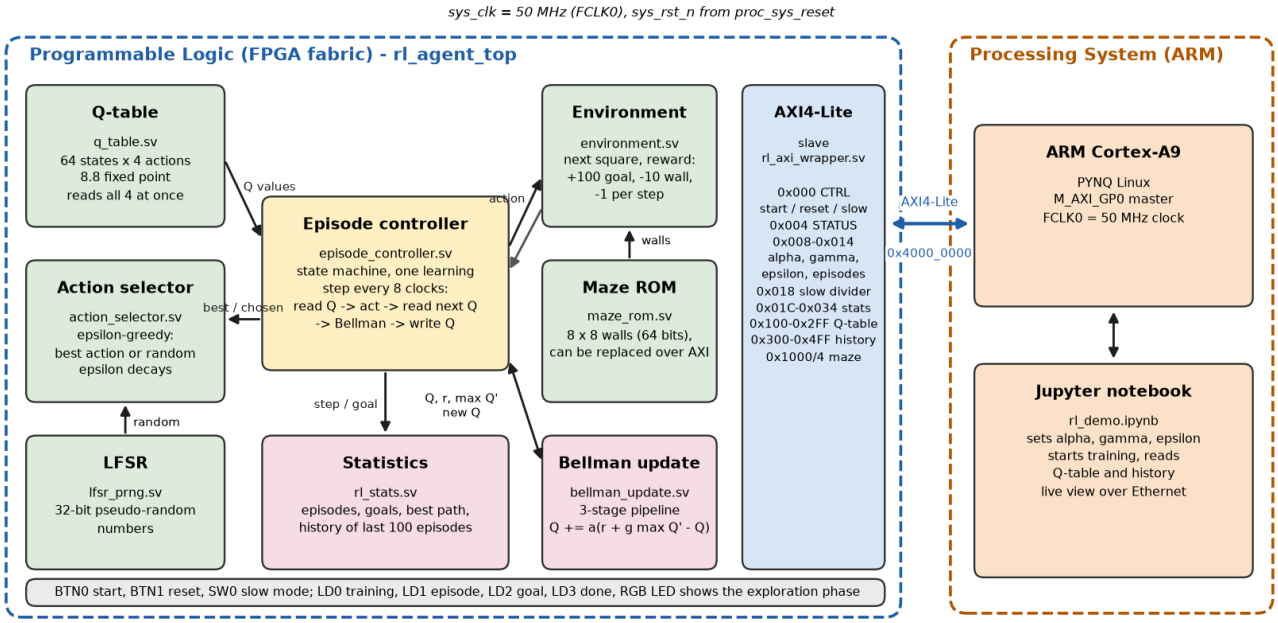


Figure 1 – System block diagram

The ARM processor writes hyperparameters (and optionally a new maze layout) into the AXI registers, triggers training, and reads the results. All learning computation happens in the fabric at 50 MHz; the ARM only observes.

4. RTL Design

4.1 Module hierarchy

```
system_wrapper                   (Vivado block design, FCLK0 = 50 MHz)
 +-- rl_agent_top_0             rl_agent_top.sv  (buttons, LEDs, RGB indicator)
      |-- u_maze_rom            maze_rom.sv
      |-- u_lfsr_prng           lfsr_prng.sv
      |-- u_q_table             q_table.sv
      |-- u_action_selector     action_selector.sv
      |-- u_environment         environment.sv
      |-- u_bellman_update      bellman_update.sv  (3-stage pipeline)
      |-- u_rl_stats            rl_stats.sv
      |-- u_episode_controller  episode_controller.sv  (11-state FSM)
      +-- u_rl_axi_wrapper      rl_axi_wrapper.sv
```

4.2 Modules

Module (file)	What it does
maze_rom	Stores the 8×8 maze layout as a 64-bit register (one bit per cell: 0 = open, 1 = wall). The ARM can write a new maze layout at runtime through the AXI registers.
lfsr_prng	A 32-bit linear feedback shift register (seed 0x5A8E1247). It gives a 16-bit random number for the explore/exploit decision and a 2-bit random action.
q_table	64 states × 4 actions, each a signed 8.8 fixed-point value. Supports single-cycle read of all four actions for one state, single-action write, and a 64-cycle sweep to clear the entire table on reset.
action_selector	Compares the four Q-values to find the best action and its Q-value (argmax). Compares a random 16-bit number against the current epsilon threshold to decide explore vs. exploit. After each episode epsilon is multiplied by 0.995 until it reaches the minimum of 0.05.
environment	Pure combinational: given the current position and action, checks the maze layout for walls and boundaries, computes the next position and the reward (+100 goal, −10 wall, −1 step).
bellman_update	3-stage pipeline implementing $Q_{new} = Q_{old} + \alpha \cdot (\text{reward} + \gamma \cdot \max Q(s') - Q_{old})$. Uses signed fixed-point arithmetic with saturation clamping to [−128.0, +127.996].
rl_stats	Counts episodes, steps, total reward, best path length, and total goals reached. Maintains a circular buffer of the last 100 episodes (goal flag + step count), readable from AXI.
episode_controller	11-state FSM that runs one learning step every 8 clock cycles: read Q-values, select action, query environment, read next-state Q-values, run Bellman update, write updated Q, check if the episode is done (goal, wall or 256 steps). In slow mode it waits between steps so the agent can be watched live.
rl_axi_wrapper	AXI4-Lite slave: control (start, reset, slow mode), status, hyperparameters, statistics, Q-table readout (128 words, 2 values each), episode history buffer (100 entries), and maze load registers. Total address space: 0x2000.
rl_agent_top	Top-level wrapper: instantiates all modules, synchronises the push buttons, drives the four LEDs and the RGB LED, and combines AXI and button controls.

4.3 Register map (base address 0x4000_0000)

Offset	Name	Access	Meaning
0x000	CTRL	W	bit 0 start, bit 1 reset, bit 2 slow mode
0x004	STATUS	R	bit 0 training, bit 1 finished, bits [31:16] episode number
0x008	ALPHA	R/W	learning rate (0.8 unsigned fixed-point, default 0x1A)
0x00C	GAMMA	R/W	discount factor (0.8 unsigned fixed-point, default 0xF3)
0x010	EPSILON	R/W	initial exploration rate (0.8 unsigned fixed-point, default 0xE6)
0x014	MAX_EPISODES	R/W	number of episodes to train (default 2000)
0x018	SLOW_DIVIDER	R/W	clock cycles per step in slow mode (default 2,500,000 = 50 ms)
0x01C–0x034	STATS	R	episode, steps, reward, position, best path, epsilon, total goals
0x100–0x2FF	Q_TABLE	R	128 words: 2 signed 8.8 Q-values per word (UP/DOWN then LEFT/RIGHT)
0x300–0x4FF	HISTORY	R	100 entries: bit 31 = goal reached, bits [23:0] = step count
0x1000	MAZE_LO	R/W	lower 32 bits of the 64-bit maze layout
0x1004	MAZE_HI	R/W	upper 32 bits; writing this register loads the new maze

4.4 Board LEDs and controls

I/O	Function
LD0	Training is running
LD1	Toggles at the end of every episode
LD2	Flashes for ~100 ms when the agent reaches the goal
LD3	Training has finished
RGB LD4	Red (exploring, $\varepsilon > 0.60$), yellow = red + green (balanced, $0.20 < \varepsilon \leq 0.60$), green (exploiting, $\varepsilon \leq 0.20$). See the note below about the pins.
BTN0	Start training
BTN1	Reset everything
SW0	Slow mode (one step every 50 ms for the live maze view)

Note on the RGB LED pins. The first constraints file mapped rgb_led0 to pins M15, G14 and L15. On the PYNQ-Z2, M15 is the red and G14 the blue of LD5, so with that bitstream the colour appeared on LD5 as red → magenta → blue. The pins were corrected to N15, G17 and L15 (LD4 red, green, blue) and the bitstream was rebuilt on 2026-09-19 (timing met, numbers in section 6.1), so LD4 now shows red → yellow → green. The board results in section 6.2 were taken with the earlier bitstream; the change only moves the LED pins and does not touch the learning logic.

4.5 RTL schematics

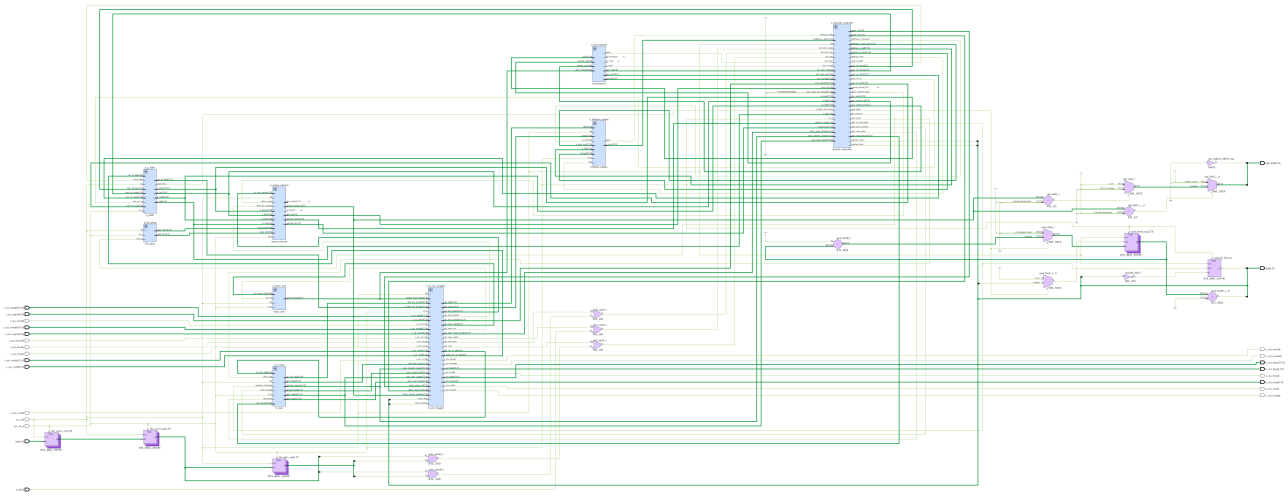


Figure 2 – Elaborated RTL schematic of rl_agent_top (Vivado)

5. Simulation Results

The design was simulated with Icarus Verilog 12 using the testbench *rl_agent_tb.sv*, which acts as the ARM processor over AXI4-Lite and trains the agent for the full 2000 episodes on the built-in maze (details in Simulation/simulation_report.pdf). All checks passed:

#	Check	Result
1	Default settings (alpha 0x1A, gamma 0xF3, 2000 episodes)	**PASS**
2	Training runs all 2000 episodes and reaches the goal	**PASS** – 1335 goals
3	Best path is the 14-step shortest path	**PASS** – 14 steps
4	Move into the goal learned, Q((7,6), RIGHT) > 80	**PASS** – 96.37

```
finished: 2000 episodes, 1335 goals, best path 14 steps
[PASS] the agent found the shortest path through the maze
Q((7,6), RIGHT) = 96.37
[PASS] the value of stepping into the goal was learned
history entry 0: goal 1, 14 steps
PASS: the hardware agent learned to solve the maze
```

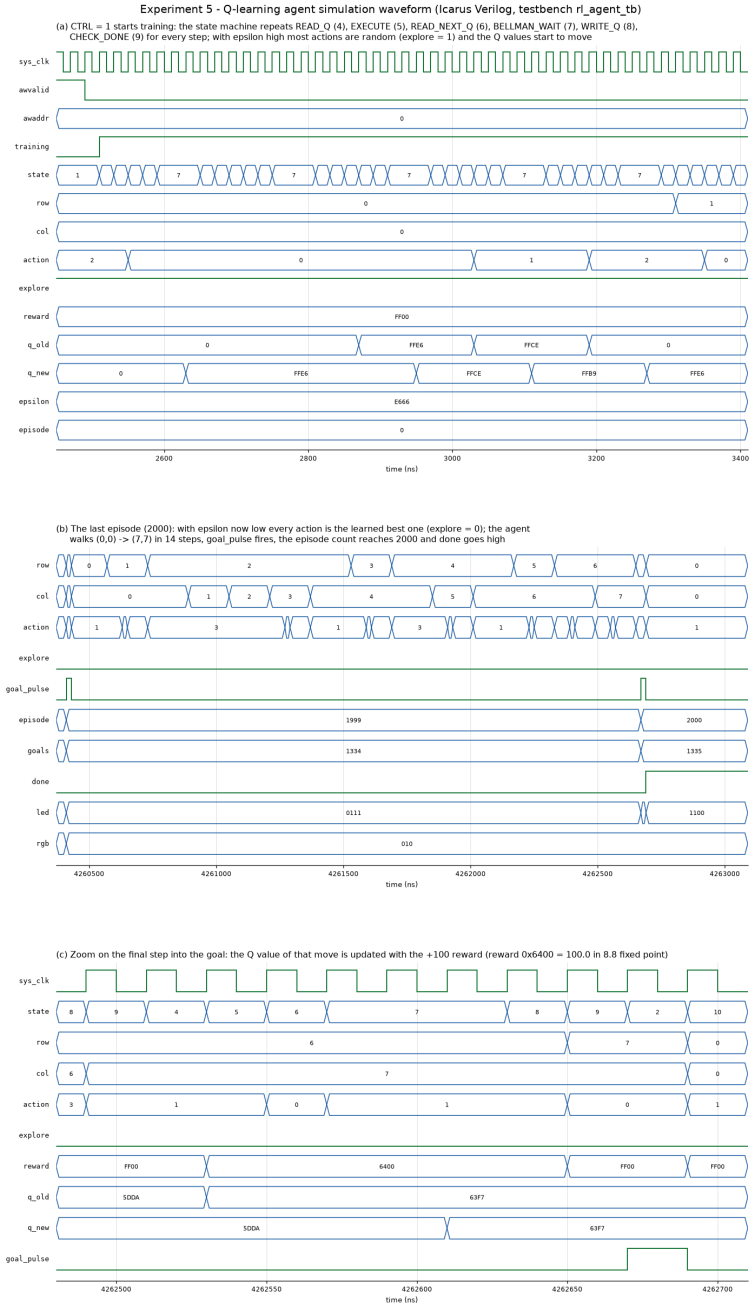


Figure 3 – Simulation waveform: start of training, the last episode and the final step into the goal

6. Hardware Implementation

6.1 Build

The design was built with Vivado 2026.1 for the xc7z020clg400-1 device using build.tcl, with the fabric clock set to 50 MHz via the processing system's FCLK_CLK0. The constraints file (pynq_z2.xdc) assigns the four LEDs, the RGB LED, the two buttons and the switch. The Q-table (64 states × 4 actions × 16 bits = 4,096 bits) is written one entry at a time but all four values of a state are read at once, so Vivado builds it from flip-flops and multiplexers instead of block RAM. That is why the Q-table uses most of the registers and LUTs, and no block RAM is used.

Resource	Used	Available	Utilisation
Slice LUTs	6,576	53,200	12.36 %
of which LUT as memory (shift registers)	60	17,400	0.34 %
Slice registers (flip-flops)	7,876	106,400	7.40 %
Block RAM tiles	0	140	0.00 %
DSP48 blocks	3	220	1.36 %
Bonded I/O pins	10	125	8.00 %
Global clock buffers	1	32	3.13 %

Timing (50 MHz clock)	Value	Status
Worst setup slack (WNS)	+2.473 ns	**met**
Worst hold slack (WHS)	+0.049 ns	**met**
Timing endpoints	16,603	

With a WNS of +2.473 ns, the design has 2.5 ns of positive slack and could run at approximately 57 MHz if needed.

6.2 Test on the board

The steps of the notebook *rl_demo.ipynb* (Jupyter folder) were run on the PYNQ-Z2 with PYNQ (tools/execute_all_on_pynq.py runs them on the board over SSH and copies the log and graphs back; tools/embed_nb_outputs.py puts them into the notebook). The run loads the bitstream, trains the agent for 2000 episodes with the default settings, reads the statistics, the Q-table and the episode history over AXI, runs one episode in slow mode for the live view, and then loads three new mazes over AXI and trains on each of them. The log of the run and the graphs it drew are in Jupyter/output. The numbers below are taken from that log and compared with the simulation by tools/p5_rl_results.py.

Check	Detail	Result
Training ran all episodes	2000 episodes, 1335 reached the goal	**PASS**
Shortest path found	14 steps (shortest possible 14)	**PASS**
Board result equals simulation	1335 goals on the board, 1335 in simulation	**PASS**
Live run reaches the goal	slow-mode episode found the goal in 14 steps	**PASS**
New mazes loaded over AXI	Corridor Detour: 14, Center Barrier: 14, Zig-Zag Staircase: 14	**PASS**

The board gives exactly the same result as the simulation (1335 goals, best path 14). This is expected: the LFSR starts from the same seed, so the hardware makes the same random choices in the same order.

Board log (Jupyter/output/terminal_output.txt)

```
[*] Flashing FPGA bitstream for Experiment 5 (RL Agent)...
[+] FPGA Flashed! DONE LED active.
Overlay loaded, agent at 0x40000000
episodes      : 2000
goals reached : 1335
best path     : 14 steps (shortest possible is 14)
final epsilon : 0.050
measured 4.9 ms for 2000 episodes on hardware
[Ethernet Visualizer] Streaming live learning dynamics over Ethernet (8.0s)...
[Ethernet Visualizer] GOAL REACHED! Target found in 14 steps! Total goals: 1336
[+] Saved live visualizer frame: /tmp/pynq_runs/exp5_graph_2_live_visualizer_frame.jpg
Maze 'Corridor Detour': best path = 14 steps
Maze 'Center Barrier': best path = 14 steps
Maze 'Zig-Zag Staircase': best path = 14 steps
```



Figure 4 – Results of rl_demo.ipynb on the PYNQ-Z2: learned policy, three new mazes and the live view

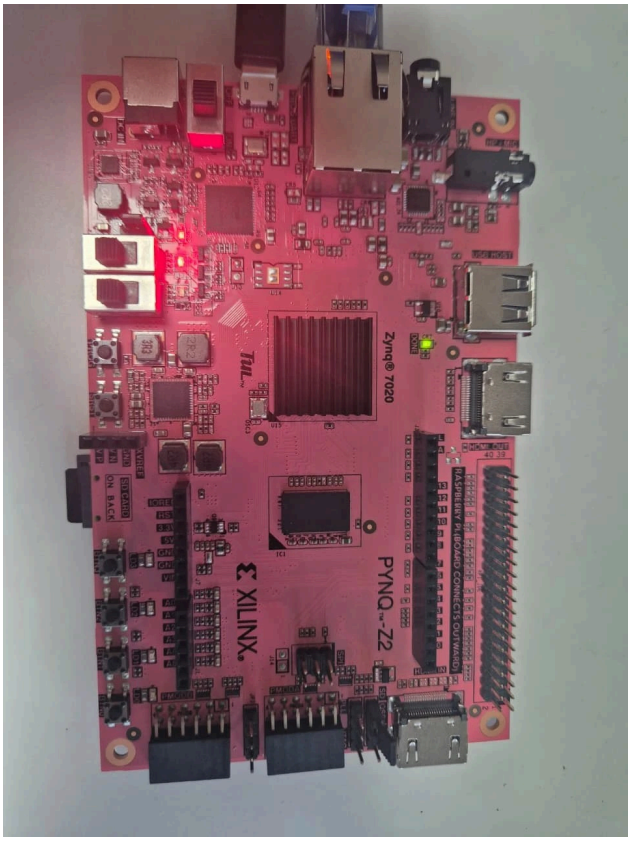


Figure 5 – PYNQ-Z2 board setup: micro-USB (power and JTAG/UART), Ethernet to the PC, PYNQ Linux on the SD card

6.3 Learning progress

The simulation transcript prints the goal count every few episodes. It shows the usual three phases of ε-greedy Q-learning (the board run gives the same totals):

Phase	Episodes	Goals reached	What happens
Exploring	0 – ~360	0	ε is high, almost every action is random and most episodes end at a wall
Transition	~360 – ~500	60 by episode 496	the goal is found and its value spreads back through the Q-table
Exploiting	~500 – 2000	1335 by episode 2000	about 80 % of episodes reach the goal; ε settles at 0.05

In the last episode (Figure 3b) the agent follows the learned policy and walks the 14-step shortest path. The three extra mazes (Corridor Detour, Center Barrier and Zig-Zag Staircase) were loaded over AXI without a new bitstream, and the agent found the 14-step shortest path in each of them.

6.4 Training speed

At 50 MHz one learning step takes 8 clock cycles (160 ns). An episode ends at the goal, at a wall or after 256 steps, so the early exploring episodes are short. The whole 2000-episode training took 4.4 ms of simulated time, and 4.9 ms was measured on the board from Python (this includes the time Python spends polling the status register). A Python Q-learning loop on the ARM core needs several microseconds for each step, so the hardware is much faster.

7. Applications

- **Autonomous navigation:** a hardware RL agent at the edge of an embedded system learns to navigate physical environments (e.g. a robot in a warehouse grid) in real time, without waiting for a cloud server.
- **Adaptive control:** the Q-table can be mapped to sensor readings and actuator commands, so the agent learns to optimise power management, thermal regulation, or network routing directly in hardware.
- **Hardware-accelerated AI research:** running RL episodes in the FPGA fabric is much faster than software, so many hyperparameter settings can be tried quickly.
- **Teaching and demonstration:** the LEDs and RGB indicator make the learning process visible, and the live maze view in slow mode lets students watch the agent explore and exploit in real time.

8. Conclusion

A complete Q-learning reinforcement learning agent was designed, simulated and implemented in the FPGA fabric of the PYNQ-Z2. The agent learns to solve an 8×8 maze from scratch with no prior knowledge, finds the optimal 14-step path, and trains 2000 episodes in under 5 ms. All simulation and board checks passed, the board result is identical to the simulation, and three new mazes loaded at run time were also solved. The only issue found, the RGB LED pin mapping, was fixed in a rebuilt bitstream (section 4.4).

Learning outcomes: the Bellman equation and how to implement it in fixed-point arithmetic; epsilon-greedy exploration and how exponential decay balances exploration with exploitation; designing a multi-state FSM that coordinates pipelined arithmetic with memory reads and writes; building a complete AXI4-Lite register file with read-only statistics, read-write configuration, and memory-mapped table readout; and using the PYNQ framework to interact with custom hardware from Python.

Future scope: implement Deep Q-Networks (DQN) with a small neural network in the fabric instead of the flat Q-table; add SARSA or other on-policy algorithms for comparison; support larger mazes with block RAM storage; add HDMI output for real-time maze visualisation without the notebook; and implement multi-agent RL with several agents sharing the same maze.